

Proof Buddy Testing Guide

This document explains how every automated test in the repository is run, and briefly describes what each test file covers.

Manual browser smoke tests for equational reasoning live in [TESTING_MANUAL_QA.md](#). Older notes that only cover the [proofs/](#) engine modules live in [django_server/proofs/TEST_STRUCTURE.md](#).

Overview

Proof Buddy has four kinds of tests:

Kind	Runner	Needs a live server?	Needs MySQL?
Django <code>TestCase</code> / <code>APITestCase</code> modules	<code>python manage.py test</code>	No	Yes (creates a separate <code>test_proofbuddy</code> database)
Import-time Python scripts (no <code>TestCase</code> classes)	Imported by Django discovery, or run as <code>python -m ...</code>	No	Only if Django discovery imports them
Jest / React Testing Library	<code>npm test</code> in <code>client/</code>	No	No
Live-server scripts (HTTP or Puppeteer)	Run the file directly	Yes	Yes (the real app database)

There is no CI pipeline. Tests are run locally.

Prerequisites

- Python virtualenv activated, with `django_server/requirements.txt` installed
- MySQL running, and a project-root `.env` with `DB_*` values
- Node.js / npm, with `client/node_modules` installed (`cd client && npm install`)

Django tests use a **separate** MySQL database named `test_proofbuddy` (`DATABASES['default']['TEST']['NAME']` in `django_server/django_server/settings.py`). They should not write to the development database.

Windows encoding

Several tests print Unicode rule text (↔). In PowerShell, set this before Django test commands:

```
$env:PYTHONIOENCODING = "utf-8"
```

Then run tests from `django_server/`:

```
cd django_server
py manage.py test
```

How to run the suites

Run almost everything (expression engine + Django + Jest)

From `django_server/`:

```
$env:PYTHONIOENCODING = "utf-8"
py -m expression_tree.runTests
```

`expression_tree/runTests.py` does three things in order:

1. Imports `expression_tree.testApplyRule` and `expression_tree.testAdvMath` (they run on import)
2. Runs `python manage.py test --verbosity=2` (all Django apps)
3. Runs `npx jest --no-coverage` in `client/`

It does **not** run the live-server scripts (`test_equational_api.py`, `trial-automation.js`) or the skipped `expression_tree` files.

Backend — all Django tests

From `django_server/`:

```
$env:PYTHONIOENCODING = "utf-8"
py manage.py test
```

Django discovers `test*.py` and `tests.py` under installed apps (`accounts`, `racket_api`, `proofs`, `induction_api`, `equational_reasoning_api`, `assignments`). The custom runner `proofs.runner.ColorDiscoverRunner` only adds red FAIL/ERROR coloring.

Run one app or one module:

```
py manage.py test proofs
py manage.py test induction_api
py manage.py test equational_reasoning_api
py manage.py test assignments
py manage.py test racket_api

py manage.py test induction_api.test_check_complete
py manage.py test equational_reasoning_api.test_value_mapping
```

Useful flags: `--verbosity=2`, `--keepdb` (reuse `test_proofbuddy` instead of recreating it).

Backend — expression-tree scripts (not Django TestCases)

These files assert by printing PASS/FAIL as they execute. They are **not** collected as Django tests unless some other Django module imports them.

From `django_server/`:

```
py -m expression_tree.testApplyRule
py -m expression_tree.testAdvMath
py -m expression_tree.test_rule_subclasses
py -m expression_tree.test_rule_edge_cases
```

`py -m pytest expression_tree/testApplyRule.py` appears in older docs. **pytest is not a project dependency**. Prefer the `python -m` commands above.

Frontend — Jest

From `client/`:

```
npm test
```

That script runs Jest with a larger Node heap (`NODE_OPTIONS=--max-old-space-size=4096`). Config is `client/jest.config.js` (jsdom, Babel, CSS/image mocks, `src/setupTests.js`).

```
npm test -- --watch
npm test -- --coverage
npm test -- --testPathPattern=InductionRacket
npm test -- --testPathPattern=playMode
npm test -- --testPathPattern=EquationalReasoningNew
```

Jest picks up `*.test.js` files. Frontend tests mock HTTP and layout, so the Django server does **not** need to be running.

Live-server / browser scripts

These hit a running app. Start Django (`localhost:8000`) and, for Puppeteer, the React dev server (`localhost:3000`).

```
# From repo root — HTTP smoke of equational endpoints (auth is not configured in
the script)
py test_equational_api.py
```

```
# From repo root – start both servers, then drive induction UI with Puppeteer
.\trial.ps1
```

`trial.ps1` launches the two servers and then `node trial-automation.js`. Puppeteer is a root `package.json` dependency (`npm install` at the repo root if needed).

Manual browser QA

Follow [TESTING_GUIDE.md](#) against <http://localhost:3000/equational-reasoning>. Course UI checks are listed in [COURSE_DEVELOPER_NOTES.md](#).

How Django discovery interacts with import-time tests

Several `proofs/test_*.py` files are **not** `django.test.TestCase` classes. They run their assertions at **import time**. When Django discovers `proofs/`, it imports those modules, so the checks still execute, even though Django may report “Ran 0 tests” for that file.

`proofs/tests.py` is only a short comment; it no longer orchestrates other modules. Discovery plus `ColorDiscoverRunner` is the runner.

`django_server/test_manual_persistence.py` sits next to `manage.py`, not inside an app. `py manage.py test` (no label) will **not** pick it up. Run it explicitly:

```
py manage.py test test_manual_persistence
```

Frontend test files

Support files (not test suites): `client/src/setupTests.js` (jest-dom, `ResizeObserver` and `sessionStorage` stubs) and `client/src/__mocks__/fileMock.js` (static-asset stub).

File	What it tests
<code>client/src/App.test.js</code>	Smoke: <code>App</code> renders without throwing.
<code>client/src/test/EquationalReasoningNew.test.js</code>	Equational Reasoning page UI with services and heavy children mocked: headings, name/tag/goal fields, typing, start-form validation (missing name/goals, identical LHS/RHS, reserved names), and the start confirmation modal.
<code>client/src/test/InductionRacket.test.js</code>	Induction page UI with the same mock pattern: headings, integer/list radios, induction parameters, goal fields, typing, validation (anchor value, leap vs induction

File	What it tests
	variable, leap in a goal, missing IH, reserved names), and the start confirmation modal.
<code>client/src/test/playMode.test.js</code>	Pure unit tests of <code>playModeUtils.js</code> (no React): init, advance, cancel, visibility, Continue-button logic, and independence of induction case/side slots. Also includes a manual UI checklist in comments.
<code>client/src/test/playModeFeatures.test.js</code>	Play Mode UI: "Run Proof" on proof cards, premise-only edge case, footer greying vs Continue, row-number click blocking, full advance/cancel lifecycles, and independent base/leap × LHS/RHS state.

Django app: `proofs/` (expression engine, mostly import-time)

File	What it tests
<code>proofs/tests.py</code>	Not a suite. Documents that Django discovery plus <code>ColorDiscoverRunner</code> is the runner.
<code>proofs/test_helpers.py</code>	Shared helpers (<code>do_single_test_case</code> , <code>test_racket_function</code> , <code>test_axiom</code> , node-id utilities). Not a suite by itself.
<code>proofs/test_math_operations.py</code>	Import-time: <code>+</code> , <code>-</code> , <code>*</code> , <code>quotient</code> , <code>remainder</code> , <code>expt</code> and comparisons; arity, types, generics, unresolved args.
<code>proofs/test_logic_operations.py</code>	Import-time: <code>not</code> , <code>and</code> , <code>or</code> , <code>xor</code> , <code>implies</code> and error cases.
<code>proofs/test_list_operations.py</code>	Import-time: <code>cons</code> , <code>first</code> , <code>rest</code> and predicates (<code>zero?</code> , <code>null?</code> , <code>if</code> , <code>integer?</code> , <code>list?</code>).
<code>proofs/test_axioms_and_udfs.py</code>	Import-time: axiom parameter mapping, built-in axioms, and UDF definition/application.
<code>proofs/test_integration.py</code>	Import-time: node methods, JSON/position helpers, proof-building and math-rewrite demos.
<code>proofs/test_induction.py</code>	Import-time: a full integer induction proof (base + leap, both sides), driven by <code>proofs/indTest.txt</code> .
<code>proofs/test_lemma_application.py</code>	Import-time, no DB: <code>LemmaRule</code> / <code>LemmaApplicator</code> (param counts, names, highlight, happy paths).
<code>proofs/test_udf_in_udf.py</code>	Import-time: UDFs that call other UDFs (e.g. <code>rev</code> calling <code>append</code>) use the callee's signature.
<code>proofs/test_udf_if_type_mismatch.py</code>	Django <code>TestCase</code> : UDF <code>if</code> branches with mismatched types are rejected; <code>?</code> in UDF labels.

File	What it tests
proofs/indTest.txt	Input data for <code>test_induction.py</code> , not a test file.
proofs/runner.py	Custom Django test runner (colorized FAIL/ERROR). Not a test file.

Django app: `equational_reasoning_api/`

File	What it tests
<code>equational_reasoning_api/tests.py</code>	<code>EquationalProof</code> / <code>EquationalProofLine</code> models and serializer validation (empty or identical goals).
<code>equational_reasoning_api/test_integration.py</code>	Authenticated HTTP walkthrough of the main equational endpoints.
<code>equational_reasoning_api/test_set_parameters.py</code>	Support-parameter fields on the model and <code>PATCH /set-parameters</code> (instructor vs student, get-proof-lines payload).
<code>equational_reasoning_api/test_value_mapping.py</code>	High vs low support: infer UDF/axiom/lemma parameters from a highlight; engine tests plus HTTP.
<code>equational_reasoning_api/test_rewrite_logic.py</code>	Engine-only <code>rewrite logic</code> (<code>AdvLogic</code>) equivalences and rejections.
<code>equational_reasoning_api/test_hidden_expression.py</code>	Hidden definition bodies: model/serializer, helpers, student validation, masking on get-proof-lines.
<code>equational_reasoning_api/test_rule_comparison.py</code>	<code>normalize_rule_string</code> / <code>compare_rule</code> for hidden-field checks (<code>with</code> , <code>⇒</code> vs <code>=</code> , high-support bare <code>apply F</code>).

Django app: `induction_api/`

File	What it tests
<code>induction_api/tests.py</code>	Create/start/clear proofs, list (some list cases skipped), engine endpoints (<code>set-current-proof</code> , <code>apply-rule</code> , <code>check-goal</code> , <code>delete-line</code> , substitution), URL registration.

File	What it tests
<code>induction_api/test_check_complete.py</code>	<code>TwoSidedProof.checkComplete()</code> ignores trailing blank (cleared) lines.
<code>induction_api/test_set_parameters.py</code>	Induction support-parameter fields and <code>PATCH /set-parameters</code> .
<code>induction_api/test_proof_management.py</code>	Clear / archive / restore induction proofs.
<code>induction_api/test_proof_line_persistence.py</code>	Proof lines persist with the expected rules after API use.
<code>induction_api/test_database_persistence.py</code>	Broader line persistence, including rules written to the DB.
<code>induction_api/test_error_persistence.py</code>	Failed-rule errors stay on the correct case×side row (base vs leap isolation).
<code>induction_api/tests_is_complete_persistence.py</code>	<code>is_complete</code> defaults false, becomes true only when base and leap are done, and reverts on edit.
<code>induction_api/tests_name_conflict.py</code>	Cross-mode name uniqueness (induction vs equational), check-name-conflict endpoint, archive-on-reuse.
<code>induction_api/tests_proof_card_fields.py</code>	List endpoint includes leap goals and <code>is_complete</code> for proof cards.
<code>induction_api/test_value_mapping_rule_response.py</code>	High support: apply-rule JSON <code>rule</code> includes $\mapsto$ mappings; low support does not.
<code>induction_api/test_wrong_highlight.py</code>	Engine-only: highlighting the wrong node on a UDF apply returns a user error instead of <code>IndexError</code> .
<code>induction_api/test_lemma_param_generics.py</code>	Engine-only: lemma free vars are not dropped because they match the caller's generic names.

Django app: `assignments/`

File	What it tests
<code>assignments/tests/tests_assignment_api.py</code>	Instructors cannot create assignments on another instructor's course; deleting an assignment cascades.
<code>assignments/tests/tests_course_api.py</code>	Course term updates, invalid terms, instructor vs student list views, regenerating join codes.
<code>assignments/tests/tests_invitation_api.py</code>	Students can accept or reject course invitations.

File	What it tests
<code>assignments/tests/tests_permissions.py</code>	Students cannot open inactive courses; instructors can.
<code>assignments/tests/tests_proof_cloning.py</code>	Starting an assignment proof deep-clones the template; restarting returns the existing clone.
<code>assignments/tests/tests_roster_api.py</code>	Join codes, instructor add/remove, duplicate-email conflict, leave-course, invitation cleanup.

Django app: `racket_api/`

File	What it tests
<code>racket_api/tests.py</code>	Empty Django placeholder (<code># Create your tests here.</code>).
<code>racket_api/test_delete_definition_cache.py</code>	After permanently deleting a UDF, the same label can be created again (engine cache is cleaned). Duplicate labels are still rejected while the definition exists.

Django app: `accounts/`

File	What it tests
<code>accounts/tests.py</code>	Empty Django placeholder. Auth is not covered by an automated suite.

`expression_tree/` scripts (run with `python -m`, not `manage.py test`)

File	What it tests
<code>expression_tree/runTests.py</code>	Orchestrator: <code>testApplyRule</code> , <code>testAdvMath</code> , then Django, then Jest.
<code>expression_tree/testApplyRule.py</code>	Import-time loop: <code>ERProof.addProofLine</code> for <code>cons/first/rest/if/null?/zero?</code> and related rules. Prints before/after or <code>errLog</code> .
<code>expression_tree/testAdvMath.py</code>	<code>rewrite math</code> helpers (<code>_fresh_var</code> , <code>_collect_node_names</code> , <code>_abstractedMathStr</code>) and <code>AdvMath.isApplicable</code> .
<code>expression_tree/test_rule_subclasses.py</code>	Broad <code>ERRuleset</code> Rule subclass coverage (built-in, math, logic, UDF, IH, lemma, axiom, <code>AdvMath/AdvLogic</code> , eval vs rewrite).
<code>expression_tree/test_rule_edge_cases.py</code>	Companion robustness cases and documented gaps (bad <code>startPos</code> , <code>AdvMath</code> false positives,

File	What it tests
	shared rule-object mappings).
<code>expression_tree/testType.py.skip / .disabled</code>	Disabled type-system experiments. Not run.
<code>expression_tree/testDefinition.py.skip / .disabled</code>	Disabled definition experiments. Not run.

Standalone and non-app files

File	How to run	What it tests
<code>django_server/test_manual_persistence.py</code>	<code>py manage.py test test_manual_persistence</code>	Django <code>TestCase</code> : creating <code>InductionProofLine</code> rows and reading them back. Not on the default all-apps path.
<code>test_equational_api.py</code> (repo root)	<code>py test_equational_api.py</code> with Django running	Live HTTP smoke of six equational endpoints. No auth headers; expect 401s unless you add a token.
<code>trial-automation.js</code>	<code>node trial-automation.js</code> (servers already up)	Puppeteer: log in, open induction, enable (<code>f n</code>), fill a standard sum proof form.
<code>trial.ps1</code>	<code>.\trial.ps1</code>	Starts Django + React, then runs <code>trial-automation.js</code> .

Quick command cheat sheet

```
# Windows: UTF-8 for Django output
$env:PYTHONIOENCODING = "utf-8"

# Almost everything
cd django_server
py -m expression_tree.runTests

# Django only
```

```
py manage.py test
py manage.py test induction_api --verbosity=2
py manage.py test equational_reasoning_api.test_value_mapping

# Expression engine only
py -m expression_tree.test_rule_subclasses
py -m expression_tree.test_rule_edge_cases

# Frontend only
cd ../client
npm test
npm test -- --testPathPattern=playModeFeatures
```